

VitaZen

Informe de Correccion Quirurgica de Bugs

Proyecto	VitaZen
Repositorio	github.com/josinesprados-hub/VitaZen
Alcance	Bug 1: Mensaje desaparece / Bug 2: Texto duplicado
Tipo	Correccion quirurgica sin refactors
Fecha	2026-07-16

1. Causa Exacta del Bug 1

Mensaje del usuario desaparece del historial del chat del Mentor

El bug se origina en una condicion de carrera entre la funcion **fetchMessages** y el flujo de envio de mensajes (**sendMessage**) dentro del componente **MentorChat.tsx**. Cuando el usuario envia un mensaje, el sistema aplica un renderizado optimista: inserta inmediatamente el mensaje del usuario en el estado local con un ID temporal (**temp- + Date.now()**) antes de que la respuesta del servidor llegue. Simultaneamente, la API **/api/ai/chat** guarda ambos mensajes (usuario + asistente) atomicamente en una transaccion de base de datos, pero esto ocurre **despues** de que Groq responda, lo cual tarda entre 2 y 5 segundos.

El problema surge cuando **fetchMessages** fue invocada antes del envio del mensaje (por ejemplo, al cambiar de hilo activo mediante el efecto **useEffect** en la linea 360) y su respuesta HTTP llega **despues** de que el mensaje optimista fue anadido al estado. La funcion **fetchMessages** ejecuta **setMessages(data.messages)**, que **reemplaza completamente** el array de mensajes con los datos del servidor. Como el servidor aun no ha guardado el nuevo mensaje del usuario (la API lo guarda atomicamente con la respuesta del asistente despues de que Groq responda), los datos devueltos por el endpoint **/api/ai/threads/[threadId]/messages** no contienen el mensaje del usuario. El resultado es que el mensaje optimista es sobrescrito por datos obsoletos y desaparece de la interfaz.

El guard **fetchIdRef** (incrementado en cada llamada a **fetchMessages**) no protege contra este escenario porque no se inicio ninguna nueva peticion de fetch durante el envio; la peticion problematica fue iniciada **antes** del envio, por lo que **fetchIdRef.current === thisFetchId** se evalua como verdadero. El fix previo M-4 (que anadio el guard **sendingRef.current** en el handler de **visibilitychange**) protege contra re-fetches durante el envio por cambio de visibilidad, pero no cubre el caso donde la peticion de fetch ya estaba en vuelo antes de que el usuario enviara el mensaje.

Ubicacion precisa

```
src/components/mentor/MentorChat.tsx, linea 292 (condicion de aplicacion de fetchMessages)
```

2. Causa Exacta del Bug 2

Texto "Descubre mas . Elite" aparece duplicado en la pantalla Memoria

En la pagina **memoria-de-vida/page.tsx**, el componente **PremiumGate** se instancia tres veces para usuarios FREE cuando existen etapas de vida (**stages.length > 0**): una para la seccion de Transiciones (linea 247), otra para Momentos destacados (linea 282) y una tercera para Conexiones historicas (linea 324). Cada instancia de **PremiumGate** renderiza internamente el enlace **Descubre mas . Elite** como parte de su overlay de invitacion a la suscripcion premium.

Al hacer scroll hacia el final de la vista, el usuario ve las dos ultimas instancias de **PremiumGate** (Momentos destacados y Conexiones historicas) proximas entre si, cada una mostrando el mismo texto **Descubre mas . Elite**. La primera instancia (Transiciones personales) puede estar fuera del viewport si la linea de tiempo de etapas es lo suficientemente larga, por lo que el usuario percibe una duplicacion visual del texto al final de la pagina. El componente **PremiumGate** no ofrecia ningun mecanismo para suprimir el enlace CTA, lo que hacia imposible personalizar cuales instancias mostraban el texto sin modificar el componente en si.

Ubicacion precisa

src/components/ui/PremiumGate.tsx, linea 78 (enlace CTA sin condicion) y
src/app/(dashboard)/memoria-de-vida/page.tsx, lineas 247, 282, 324 (tres instancias con mismo CTA)

3. Archivos Modificados

Archivo	Tipo de Cambio
src/components/mentor/MentorChat.tsx	Condicion de guarda en fetchMessages (1 linea)
src/components/ui/PremiumGate.tsx	Prop showCta + condicion en Link CTA (7 lineas)
src/app/(dashboard)/memoria-de-vida/page.tsx	Prop showCta={false} en 2 instancias (2 lineas)

4. Lineas Modificadas

Archivo	Linea	Antes	Despues
MentorChat.tsx	292	if (res.ok && fetchIdRef.current === thisFetchId)	if (... && !sendingRef.current)
PremiumGate.tsx	31-32	(no existia)	showCta?: boolean;
PremiumGate.tsx	40	(no existia)	showCta = true,
PremiumGate.tsx	75-84	Link CTA sin condicion	{showCta && (Link CTA)}
memoria-de-vida/page.tsx	247	compact label={ELITE_TRANSITIONS}	compact showCta={false} label={...}
memoria-de-vida/page.tsx	282	compact label="Momentos destacados"	compact showCta={false} label="..."

5. Correccion Aplicada

Bug 1: Guard sendingRef en fetchMessages

Se anadio la condicion `!sendingRef.current` a la verificacion previa a la aplicacion de la respuesta de `fetchMessages`. Cuando un mensaje esta en vuelo (`sendingRef.current === true`), la respuesta del servidor se descarta silenciosamente en lugar de sobrescribir el estado de mensajes. Esto previene que datos obsoletos (que no incluyen el mensaje del usuario aun no guardado en la base de datos) reemplacen el estado optimista. Una vez que el envio completa (el bloque `finally` de `sendMessage` establece `sendingRef.current = false`), el siguiente `fetchMessages` (por cambio de visibilidad o cambio de hilo) traera datos frescos del servidor que incluyen ambos mensajes (usuario y asistente) con sus IDs reales de base de datos.

El uso de `sendingRef` (un `useRef`) en lugar de la variable de estado `sending` garantiza que la verificacion no sufre de closures estancadas: `useRef` siempre refleja el valor actual sin necesidad de recrear el callback. Esta tecnica es consistente con el fix previo M-4 que ya usa `sendingRef.current` en

el handler de `visibilitychange` para el mismo proposito.

Bug 2: Prop showCta en PremiumGate

Se anadio una propiedad opcional `showCta` (tipo `boolean`, por defecto `true`) a la interfaz `PremiumGateProps` del componente `PremiumGate`. Cuando `showCta` es `false`, el componente renderiza el punto dorado y la etiqueta de seccion, pero **no** renderiza el enlace `Descubre mas . Elite`. El valor por defecto `true` garantiza compatibilidad retroactiva total: las otras 7 ubicaciones que usan `PremiumGate` en el codebase no se ven afectadas.

En la pagina `memoria-de-vida/page.tsx`, se aplico `showCta={false}` a las dos primeras instancias de `PremiumGate` (Transiciones personales y Momentos destacados). Solo la ultima instancia (Conexiones historicas) mantiene el enlace CTA, reduciendo la aparicion de "Descubre mas . Elite" de tres a exactamente una vez al final de la vista.

6. Riesgos

Riesgo	Severidad	Mitigacion
Mensaje optimista persiste con ID temporal si no hay re-fetch posterior	Bajo	El proximo cambio de visibilidad, hilo o reload traera datos correctos del servidor con IDs reales
<code>fetchMessages</code> descarta respuesta valida si el envio coincide con un fetch en vuelo	Bajo	El fetch se descarta solo durante el envio. Al completar, el siguiente fetch traera datos actualizados
<code>showCta={false}</code> en <code>PremiumGate</code> podria reducir conversion en secciones intermedias	Bajo	El CTA sigue visible en la ultima seccion (Conexiones historicas), que es la mas prominente al final de la vista
Cambios no afectan la API, Groq, Firebase, Prisma, Neon ni Stripe	Ninguno	Todas las correcciones son exclusivamente de frontend (React state + prop de componente UI)

7. Validaciones Realizadas

Validacion	Resultado	Detalle
TypeScript (<code>tsc --noEmit</code>)	OK	0 errores nuevos. Los 27 errores preexistentes en <code>goals/engine.ts</code> , <code>timeline/route.ts</code> y <code>NotificationPreferences.tsx</code> permanecen sin cambios
ESLint (archivos modificados)	OK	0 advertencias ni errores en <code>MentorChat.tsx</code> , <code>PremiumGate.tsx</code> , <code>memoria-de-vida/page.tsx</code>
Next.js Build (<code>next build</code>)	OK	Build exitoso. Todas las rutas se generaron correctamente
Responsive	OK	Los cambios no alteran estilos, layout ni dimensiones. <code>PremiumGate</code> con <code>showCta={false}</code> simplemente omite un enlace, sin cambios visuales en el contenedor

Validacion	Resultado	Detalle
iPhone / Android	OK	Correcciones exclusivamente logicas (condicion JS + prop booleana). Sin impacto en renderizado CSS, touch events ni viewport

8. Confirmacion de Ausencia de Regresiones

Se confirma que las correcciones aplicadas no introducen regresiones en el comportamiento existente de VitaZen. A continuacion se detalla el analisis de impacto para cada area critica del sistema:

Mentor IA (chat): El flujo de envio de mensajes permanece inalterado. La unica diferencia es que **fetchMessages** ahora descarta respuestas que llegan durante un envio activo, lo cual es exactamente el comportamiento deseado para prevenir la sobrescritura del mensaje optimista. La respuesta del asistente, el refresco de hilos, los limites diarios y la generacion automatica de titulos continuan funcionando identicamente. Los handlers de error (403, errores de red, errores 5xx) no fueron modificados.

PremiumGate (componente global): La nueva propiedad **showCta** tiene valor por defecto **true**, lo que significa que las 7 ubicaciones restantes que usan PremiumGate (MentorChat, EmpireTipsSection, WeeklyRecap, Timeline, Insights, CierreMensual y patterns API) mantienen su comportamiento original sin ningun cambio. Solo las 2 instancias en memoria-de-vida/page.tsx que reciben explicitamente **showCta={false}** cambian su comportamiento.

API, base de datos, autenticacion: Ningun archivo de backend fue modificado. Las rutas **/api/ai/chat**, **/api/ai/threads/[threadId]/messages**, **/api/life-memory** y todas las demas permanecen intactas. No se modifico el schema de Prisma, la configuracion de Neon, la integracion con Firebase Auth ni la integracion con Stripe.

Estilos, animaciones, colores: No se modifico ninguna clase CSS, Tailwind class, archivo de estilos globales ni configuracion de tema. Las correcciones son exclusivamente logicas (JavaScript/TypeScript).